Q13: How Transformers Represent, Store, and Recall Factual Knowledge

DLS 2026 — module author: Kukla

October 2026

Learning objectives

- Explain where factual associations physically reside inside a transformer, distinguishing the roles of feed-forward (MLP) layers and attention, and how the key–value view of MLPs supports a "memory" interpretation.
- Describe how a stored fact is retrieved during inference, from subject enrichment in early-to-mid layers to the "extraction" event that moves an attribute into the residual stream for prediction.
- Critically assess the evidence for localized knowledge storage (causal tracing, model editing) and its limits (distributed representations, superposition, editing side effects).

1 Framing the question

A language model trained with next-token prediction never stores facts as explicit database rows. Instead, during training it compresses statistical regularities of text into its weights, and at inference it reconstructs an answer from those weights given a context. When we ask "how do models store knowledge," we are really asking two coupled questions: (i) which parameters encode a given association, and (ii) what computation reads those parameters to produce the answer. A useful running example is the factual triple *(subject, relation, object)*, e.g. "The Eiffel Tower is located in Paris." This tutorial surveys the mechanistic picture built up over the last few years, emphasizing that "storage" and "recall" are properties of a *computation over distributed parameters*, not of a lookup table.

2 MLP layers as key–value memories

The transformer block alternates self-attention with a position-wise feed-forward network (FFN/MLP) of the form $\mathrm{FFN}(x) = W_{\text{out}}\,\sigma(W_{\text{in}}x)$, where σ is a nonlinearity. In large models these MLP layers hold roughly two-thirds of all parameters, which already hints that much knowledge lives there rather than in attention.

Geva et al. [5] gave this intuition a precise form: each MLP can be read as a set of key–value memories. The rows of W_{in} act as *keys* that detect patterns in the incoming residual stream, and the matching columns of W_{out} act as *values* that are added back into the stream. Concretely, the activation $\sigma(W_{\text{in}}x)_i$ measures how strongly the i-th key fires for the current input; this scalar then scales the i-th value vector. The MLP output is a weighted sum of value vectors, i.e. a soft retrieval over a learned associative memory. Geva et al. [5] further showed empirically that individual keys

correspond to human-interpretable textual patterns (from shallow n-gram patterns in early layers to semantic topics in later layers), and that the induced value vectors frequently push the distribution toward tokens that complete those patterns. A follow-up analysis projected the value vectors directly into vocabulary space and argued that MLP updates *promote* concepts that are then surfaced in the output distribution [6].

This gives our first concrete answer: a fact is stored, in part, as a key–value pair (or a small bundle of them) distributed across MLP neurons. The "key" half recognizes the subject-plus-relation context; the "value" half writes the object into the residual stream.

3 Locating a specific fact: causal tracing

Knowing MLPs are memories does not tell us *which* memory holds "Eiffel Tower → Paris." Meng et al. [10] introduced *causal tracing* to localize individual facts. The idea is a controlled corruption-and-restoration experiment:

1. Run the clean prompt and record the correct-answer probability.
2. Corrupt the subject tokens' embeddings with noise, degrading the prediction.
3. Restore the clean hidden state at a single (layer, token) position and measure how much the correct answer recovers. The restoration with the largest "indirect effect" marks a site causally responsible for the fact.

The striking finding was that recovery peaks at MLP layers over the *last subject token*, in the early-to-middle layers of the network. This suggests a two-stage picture: mid-layer MLPs at the subject position enrich the representation with attributes of that subject, and later attention layers move the relevant attribute to the final position for prediction. Meng et al. [10] turned this localization into an editing method, ROME, which treats the critical MLP as a linear associative memory and performs a rank-one update to W_{out} so that the subject key maps to a new value (object). That editing a *single* matrix can reliably change a fact is strong causal evidence that the fact was, at least operationally, stored there.

4 The retrieval event: enrichment then extraction

How is the stored value actually read out? Geva et al. [7] dissected the recall process and proposed a clean division of labor. First, MLPs at the subject positions perform *subject enrichment*: across several layers they accumulate many attributes of the subject into its residual representation, building a rich bundle of "everything the model knows about the Eiffel Tower." Second, at the last position of the prompt, an *attention head extracts* the specific attribute demanded by the relation, copying it from the enriched subject representation into the position where the next token is predicted. Interventions that block this attention-mediated extraction suppress the correct answer even though the MLPs still "contain" the fact. Thus storage (MLP value) and selective recall (attention routing conditioned on the relation) are genuinely distinct mechanisms; both must fire for a correct answer.

This also explains why the same subject can produce different facts depending on the relation: the enriched representation is a superset, and the attention extraction step selects the slice relevant to the query. The residual stream serves as the shared communication channel [3] across which these reads and writes are composed.

5 Distributed, polysemantic, and superposed representations

The localized picture above is powerful but partial. A competing body of evidence stresses that representations are *distributed* and *superposed*. Elhage et al. [4] argued that networks pack more features than they have neurons by representing features as nearly-orthogonal directions, so a single neuron participates in many unrelated concepts (polysemanticity) and a single concept is spread across many neurons. Under superposition, "the neuron that stores Paris" is a misleading abstraction; what exists is a direction in activation space, recoverable only by looking across units. Sparse dictionary learning (e.g. sparse autoencoders) is one route to decompose activations back into more monosemantic features [2]. This tension matters pedagogically: causal tracing localizes the *computation* that writes a fact to a layer and token, but the *representation* of the fact can still be a distributed direction rather than a handful of dedicated neurons.

A related caution comes from Hase et al. [8], who showed that where a fact can be *edited* does not necessarily coincide with where causal tracing says it is *stored*; successful edits can be made at layers far from the traced site. This decouples "localization of storage" from "effective intervention point" and warns against over-reading any single localization method.

6 Evidence from model editing and its side effects

Model editing is both a tool for studying storage and a stress test of the localized hypothesis. ROME edits one fact via a rank-one MLP update [10]; MEMIT scales this to thousands of facts by spreading updates across a range of MLP layers, directly treating those layers as a batch-writable memory store [11]. The success of these methods confirms that MLP weights carry editable factual content. However, the *side effects* are instructive about how knowledge is organized. Cohen et al. [1] documented "ripple effects": editing one fact changes logically entailed facts inconsistently, revealing that related knowledge is not stored as a coherent, dereferenceable graph. Edits can also fail to generalize across paraphrases or degrade unrelated behavior, indicating that the "stored value" is entangled with the broader representation. These findings reinforce the superposition perspective: an edit nudges a direction that overlaps with many other facts.

7 Does the model "know what it knows"?

Finally, storage interacts with reliability. Even when a fact is encoded, the model may fail to recall it, or may confidently emit a plausible but false "value"—a hallucination. Kadavath et al. [9] found that models carry some internal signal of their own correctness, and that calibration and self-evaluation can partly predict whether a recalled fact is right. This suggests that recall quality is itself represented, however imperfectly, and that storage without reliable retrieval is a central failure mode of knowledge in language models.

8 Synthesis

Putting the pieces together, a defensible mechanistic story is: (1) MLP layers act as key–value associative memories [5, 6]; (2) a given fact's write-site can be causally localized to mid-layer MLPs at the subject token, and edited there [10, 11]; (3) recall proceeds by subject enrichment followed by attention-based attribute extraction at the query position [7] over the residual stream [3]; and (4) the underlying representations are nonetheless distributed and superposed [2, 4], so localization of computation must not be confused with tidy localization of representation [8], and edits ripple

through entangled knowledge [1]. Knowledge in a transformer is best understood not as a stored table but as a learned, distributed memory that is reconstructed on demand by a specific, partly interpretable computation.

References

[1] Roi Cohen, Eden Biran, Ori Yoran, Amir Globerson, and Mor Geva. Evaluating the ripple effects of knowledge editing in language models. *Transactions of the Association for Computational Linguistics*, 12:283–298, 2024.

[2] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

[3] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Christopher Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. URL https://transformer-circuits.pub/2021/framework/index.html.

[4] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. *Transformer Circuits Thread*, 2022. URL https://transformer-circuits.pub/2022/toy_model/index.html.

[5] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5484–5495, 2021.

[6] Mor Geva, Avi Caciularu, Kevin Ro Wang, and Yoav Goldberg. Transformer feed-forward layers build predictions by promoting concepts in the vocabulary space. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 30–45, 2022.

[7] Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. Dissecting recall of factual associations in auto-regressive language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 12216–12235, 2023.

[8] Peter Hase, Mohit Bansal, Been Kim, and Asma Ghandeharioun. Does localization inform editing? surprising differences in causality-based localization vs. knowledge editing in language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 17643–17668, 2023.

[9] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom

Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Christopher Olah, and Jared Kaplan. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

[10] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 17359–17372, 2022.

[11] Kevin Meng, Arnab Sen Sharma, Alex J Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.